

Modern Cloud Computing

Lecture 9: Resource Management and Scheduling II

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Classic Cluster Schedulers
 - **Quincy** (SOSP '09): scheduling as min-cost flow
 - **Delay Scheduling** (EuroSys '10): waiting for data locality
 - **Mesos** (NSDI '11): two-level scheduling with resource offers
 - **Omega** (EuroSys '13): shared state + optimistic concurrency
 - **Sparrow** (SOSP '13): fully decentralized scheduling by sampling
 - **Borg** (EuroSys '15): Google's production cluster manager

The Setting: Data-Intensive Clusters

- MapReduce (2004) triggered a wave of cluster computing frameworks: Hadoop, Dryad, Spark, MPI, Pregel, ...
- Clusters of **thousands of commodity machines**, shared by many users and jobs
- Jobs are split into many small, independent, restartable **tasks**
- Input data is stored **on the compute nodes** (GFS/HDFS): data striped across machines with direct-attached disks
- Hierarchical networks: machines → rack switch → core switch
 - Cross-rack bandwidth is scarce → **place computation near its data**

The scheduler decides where and when a task runs.

Scheduler Objectives

User Side:

- Fair sharing: my job gets its share *now*
- Low response time for short jobs
- Predictability, isolation, SLOs

Provider Side:

- High utilization
- Data locality (protect scarce network bandwidth)
- Scalability, availability, low scheduling overhead

Overview of Covered Schedulers

Year	System	Architecture	Key mechanism
2009	Quincy (Microsoft)	Centralized, monolithic	Min-cost flow optimization
2010	Delay Scheduling (Berkeley/Facebook)	Centralized, monolithic	Just wait a few seconds
2011	Mesos (Berkeley)	Two-level	Resource offers
2013	Omega (Google)	Shared-state	Optimistic concurrency
2013	Sparrow (Berkeley)	Fully distributed	Sampling + late binding
2015	Borg (Google)	Centralized (+ tricks)	Priority/quota, reclamation

Part 1: Quincy

Isard et al., Microsoft Research — SOSP 2009

Quincy: Problem & Motivation

- Dryad clusters at Microsoft: shared by tens of users, tens of concurrent jobs
- Production trace (39 days): **>50% of jobs finish in under 30 min**, some run >1 day
 - Long jobs must not starve everyone else's short jobs
- Existing schedulers (Hadoop, MapReduce): ad-hoc **queue-based heuristics** that handle the fairness–locality trade-off poorly

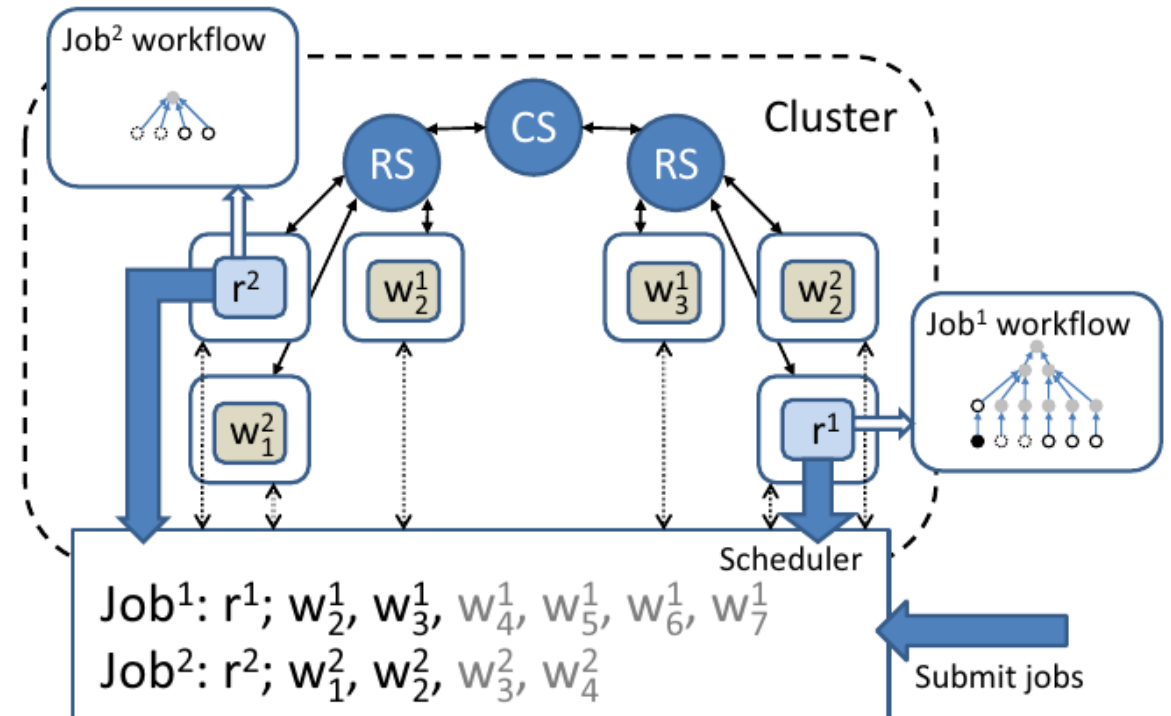
Fairness goal: with J concurrent jobs, a job that takes t seconds alone should take no more than $J \cdot t$ seconds

Quincy: Computational Model

Each job = a **root task** (coordinator) + many killable **worker tasks**; a centralized scheduler matches ready tasks to machines.

When a task becomes ready, the job reports its **preferred computers** and **preferred racks** (machines/racks storing >10% of its input); NO runtime estimates needed.

(Source: Isard et al., "Quincy," SOSP 2009, Fig. 2)



Quincy: Scheduling = Min-Cost Flow

Encode the *entire* scheduling problem as a flow network:

- Each task injects **1 unit of flow** that must reach the sink either **through a computer** (task is scheduled there) or **through an "unscheduled" node** (task waits)
- **Edge costs** encode data-transfer cost, waiting cost, and preemption cost
- **Edge capacities** encode fairness (bounds on each job's running tasks)

Solving **min-cost flow** yields the globally optimal assignment of *all* tasks of *all* jobs, simultaneously.

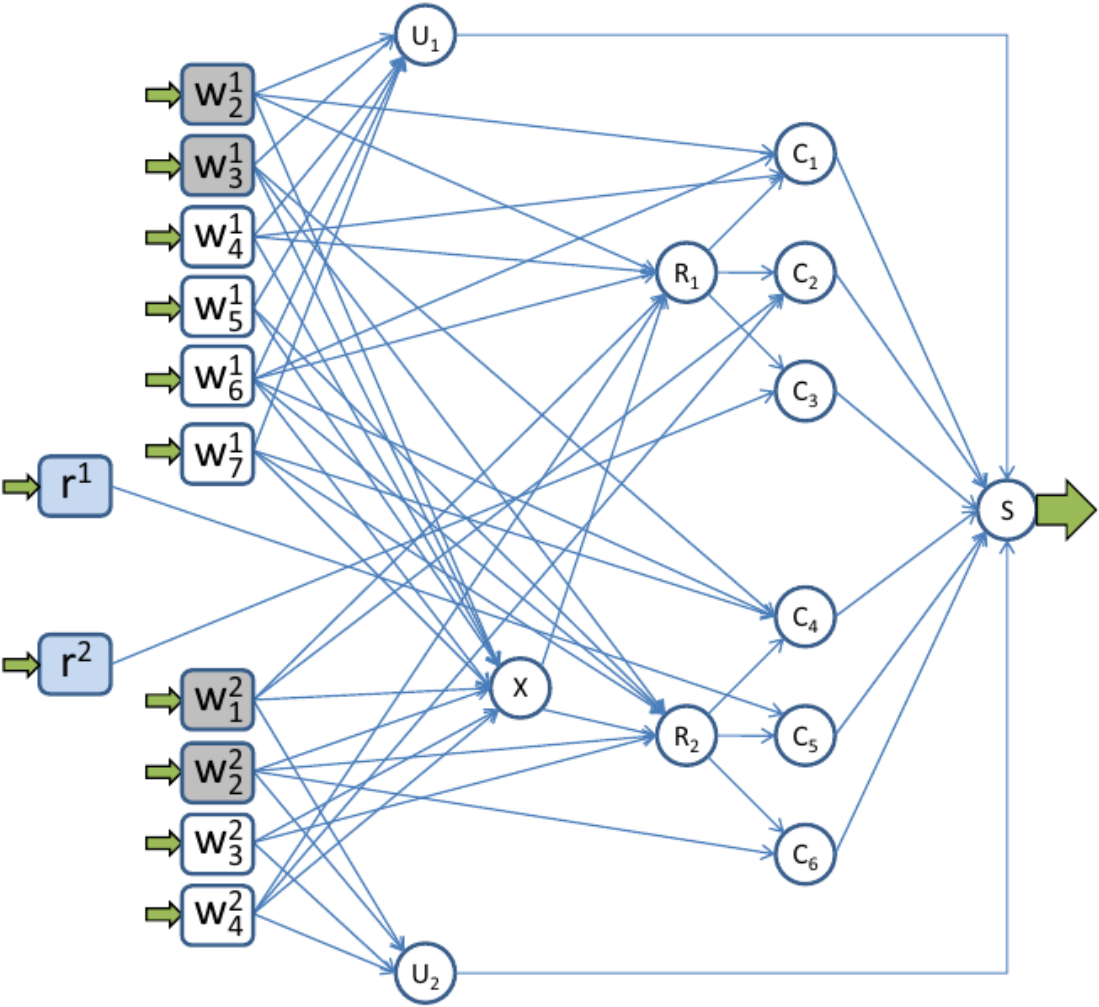
First work to model fair scheduling as min-cost flow.

The Quincy Flow Network

Symbol	Meaning
r^j	root task of job j (already running)
w_i^j	worker task i of job j (shaded = currently running)
U_j	unscheduled node for job j
C_m	computer m
R_l	rack l (rack aggregator)
X	cluster aggregator
S	sink — all flow drains here

Edge	Decision it encodes
$w_i^j \rightarrow U_j$	leave the task unscheduled (wait)
$w_i^j \rightarrow C_m$	run on a preferred computer (data is local)
$w_i^j \rightarrow R_l$	run somewhere in a preferred rack (same-rack data)
$w_i^j \rightarrow X$	run anywhere in the cluster (remote data)
$w_i^j \rightarrow C_m$ (shaded)	keep running where it already is

(Source: Quincy, SOSP 2009, Fig. 4)



Quincy: From Flow to Schedule

Reformulation

- Build a graph: tasks $\rightarrow$ {unscheduled node, preferred computers, preferred racks, cluster aggregator} $\rightarrow$ sink **S**
- Each task supplies **1 unit of flow**; all flow drains into **S**

Objective

- minimize $\sum_e cost(e) \times flow(e)$
- Subject to: flow conservation + edge capacities
- Solved with a standard min-cost flow solver

Costs: preferences

- Data transfer: local $\ll$ same-rack $\ll$ cross-cluster
- Wasted work: killing a running task costs the compute already spent
- Waiting: unscheduled-edge cost **grows with wait time** $\rightarrow$ no starvation

Capacities: constraints

- Bound how many machines a job may hold $\rightarrow$ fairness
- Bound how many tasks a machine accepts $\rightarrow$ capacity

Dynamics

- Re-solve the whole graph on every event (task finishes, node joins/fails, job arrives)
- Each solve is a fresh **global** optimum, not a per-task greedy choice
- Preemption happens when the scheduler reroutes a running task's flow

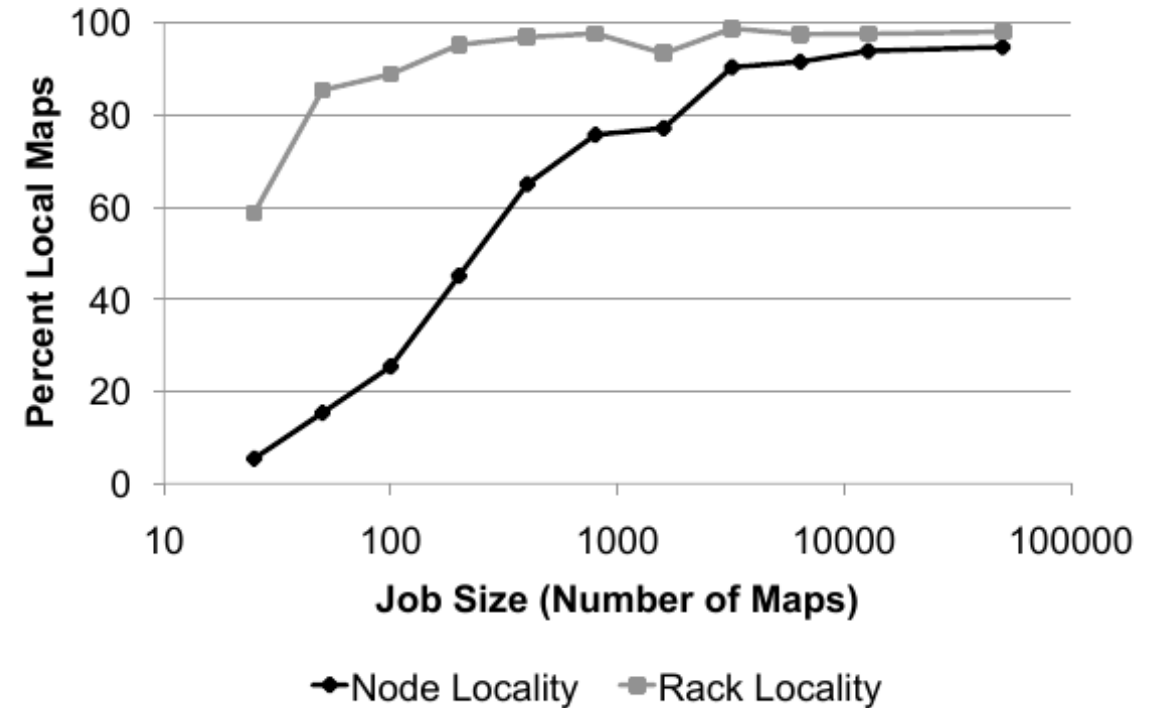
Part 2: Delay Scheduling

Zaharia et al., UC Berkeley / Facebook — EuroSys 2010

Locality vs. Job Size at Facebook

Small jobs: majority of the workload;
almost no node locality under naive fair
sharing.

(Source: Zaharia et al., "Delay Scheduling," EuroSys 2010,
Fig. 2)



Waiting

In large clusters, tasks are *short* (median task 19 s vs. median job 84 s at Facebook) and slots free up constantly (27.1 slots/second)

A job owed F slots out of S total (mean task length T) waits only $\sim F \cdot T / S$ to get its share.

Waiting will not impact job response time significantly if at least one condition holds:

- Many jobs: many jobs running, each job's fractional share of the cluster $f = F / S$ is small
- Small jobs: jobs with a small number of tasks will also have a small value of f
- Long jobs: Running time $J > T$

The Delay Scheduling Algorithm

Keep one counter per job: `skipcount` .

```
on heartbeat from node n with a free slot:
  for each job j in increasing order of running tasks:
    if j has an unlaunched LOCAL task on n:
      launch it; j.skipcount = 0
    else if j.skipcount >= D:
      launch any (non-local) task of j
    else:
      j.skipcount += 1      # skip j, try next job
```

- Skipped jobs let others go ahead for a **small, bounded time**
- The first slot offered rarely holds your data, but a slot that *does* frees up within seconds

Part 3: Mesos

Hindman et al., UC Berkeley — NSDI 2011

Mesos: Problem & Motivation

- No single framework is best for all applications: Hadoop, MPI, Dryad, Spark, Pregel, ... and new ones keep appearing
- Old solution: **statically partition** the cluster, one partition per framework
 - Low utilization; no data sharing
- Frameworks want **fine-grained** sharing (slots, short tasks: median task 23 s), but only *within* themselves

Why not one grand centralized scheduler for all frameworks?

- Would need an API expressive enough for every framework's needs
- Frameworks & policies evolve constantly (delay scheduling, LATE, ...)
- Frameworks already have sophisticated schedulers — refactoring them into a global one is impractical

Mesos: Two-Level Scheduling

Mesos is a **thin resource-sharing layer**:

- **Level 1 (Mesos master)**: decides *how many* resources each framework gets (pluggable policy: fair sharing / DRF, priorities)
- **Level 2 (framework schedulers)**: decide *which* offered resources to accept and *what tasks* to run on them

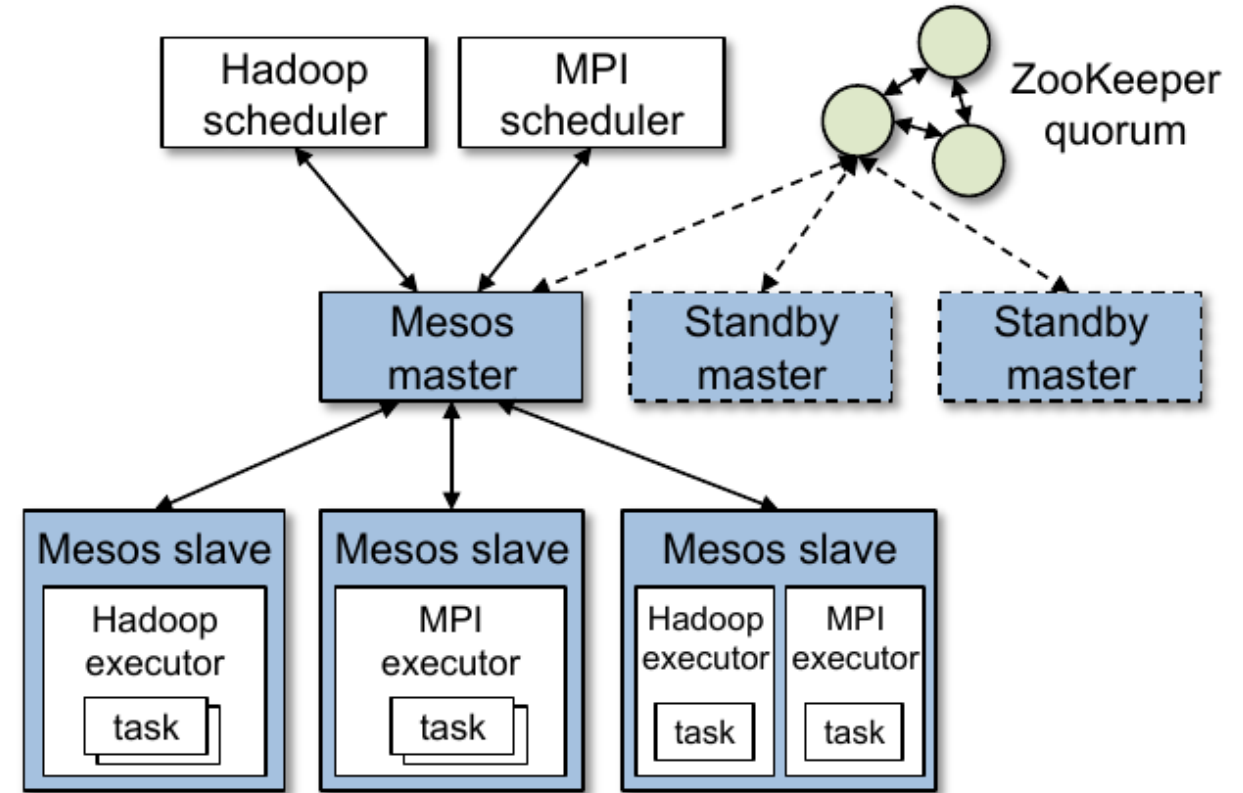
The mechanism connecting the levels: **resource offers**.

Design philosophy: keep the kernel minimal (~10 K lines of C++) and push control to frameworks — an *exokernel* for the datacenter.

Mesos Architecture

- **Master**
- **Slave daemon** per node; task isolation via **Linux containers**
- Each framework: a **scheduler** (accepts/rejects offers) + an **executor** on nodes (runs tasks)

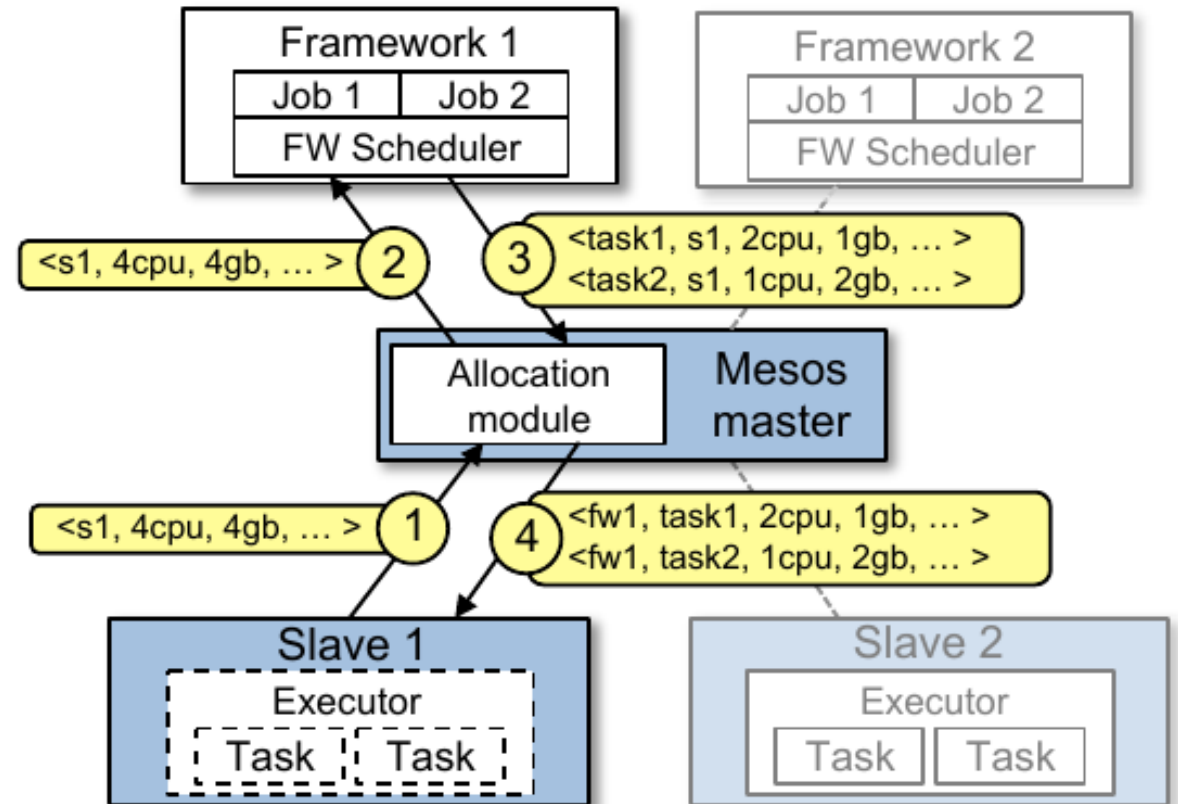
(Source: Hindman et al., "Mesos," NSDI 2011, Fig. 2)



Resource Offer Example

1. Slave reports 4 CPUs, 4 GB free;
allocation module: "*offer to framework 1*"
2. Master sends the offer $\langle s1, 4 \text{ cpu}, 4 \text{ gb} \rangle$
3. Framework's scheduler replies with tasks:
 $\langle 2 \text{ cpu}, 1 \text{ gb} \rangle + \langle 1 \text{ cpu}, 2 \text{ gb} \rangle$
4. Slave launches them; the remainder can
be offered to framework 2

(Source: Mesos, NSDI 2011, Fig. 3)



Offers vs Requests

Frameworks never tell Mesos their constraints. They just **reject** offers that don't fit and wait for ones that do.

- Arbitrarily complex framework constraints, zero complexity in Mesos
- **Filters** (e.g. "only nodes in list L") short-circuit hopeless offers
- Offered-but-unanswered resources count against your share; stale offers are **rescinded**; allocation module can **revoke** tasks (with grace period + guaranteed allocation for rigid frameworks like MPI)
- Locality without any Mesos support: frameworks apply **delay scheduling** over offers

Part 4: Omega

Flexible, Scalable Schedulers for Large Compute Clusters

Schwarzkopf et al., Google / Cambridge — EuroSys 2013

Omega: Problem & Motivation

Google's production scheduler had become **monolithic**: years of organic growth made new policies "surprisingly difficult to support in a sustainable manner."

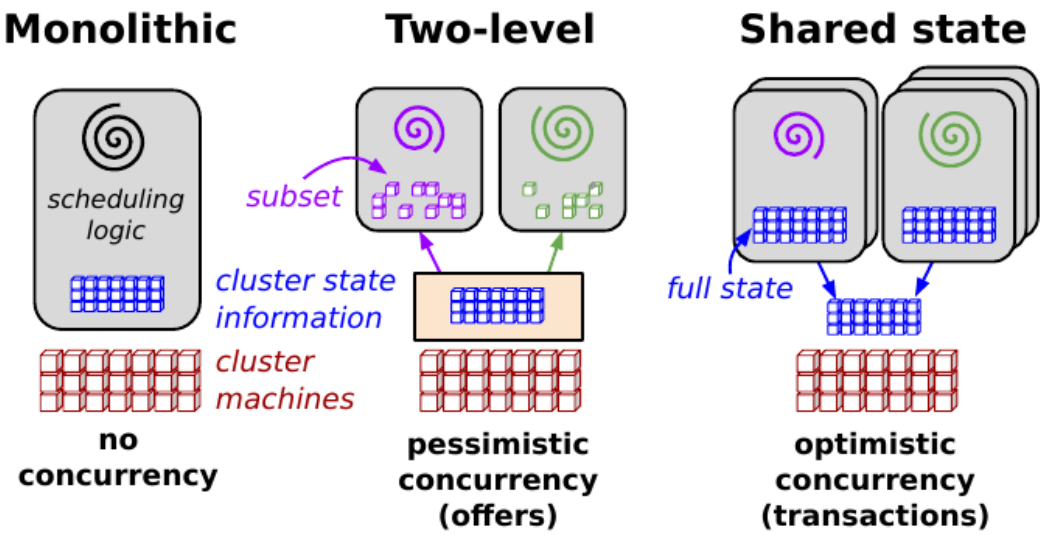
- Primary motivation is **software engineering**
- Workload is heterogeneous (three production clusters, 2011):
 - **Batch jobs**: >80% of jobs; short, frequent; want cheap fast placement
 - **Service jobs**: 20–40% run >1 month; consume **55–80% of resources**; need *careful* placement (spread across failure domains, an NP-hard optimization taking tens of seconds)
- One scheduler processing jobs one at a time → a slow service decision causes **head-of-line blocking** for the batch stream

Fix: parallelism. But which parallel architecture?

A Taxonomy of Scheduler Architectures

Approach	Visibility	Interference
Monolithic (HPC, Borg-ish, YARN)	all	none (serialized)
Static partition (Hadoop silos)	fixed subset	none
Two-level (Mesos)	offers	pessimistic
Shared-state (Omega)	all	optimistic

(Source: Omega, EuroSys 2013, Fig. 1)



Omega: Shared State + Optimistic Concurrency

- **Cell state:** a resilient master copy of *all* resource allocations in the cell
- Every scheduler keeps a **full private copy**, frequently synced
- To place a job: sync → run scheduling algorithm → submit an atomic **transaction** against the shared state
- Conflict (someone else took the resources first)? Loser resyncs and **retries** — conflicts are detected *after the fact* instead of prevented by locks
- No central resource allocator at all — it shrinks to a persistent store + validation ("ground rules" + a common precedence scale)

Schedulers run fully in parallel: **no head-of-line blocking** between batch and service.

Transactions: Incremental vs. All-or-Nothing

- **Incremental** (default): accept every non-conflicting task placement; prevents starvation → conflicts stay cheap
- **All-or-nothing**: whole job commits or nothing — this *is* gang scheduling, without Mesos-style hoarding
- Contrast with Mesos: pessimistic offers give a framework a *lock* on resources while it thinks
 - A service scheduler that ponders for tens of seconds locks the cluster's free resources away from everyone
 - Offers also hide global state

Part 5: Sparrow

Ousterhout et al., UC Berkeley — SOSP 2013

Sparrow: Problem & Motivation

Task durations keep shrinking:

- 2004: 10-minute MapReduce batch jobs → 2009: 10-s Hive queries → 2012: ~100 ms in-memory Spark / Dremel / Impala queries
- 10,000 machines × 16 cores × 100 ms tasks = **>1 million scheduling decisions/second**
- Fastest centralized schedulers: ~thousands of decisions/s (Spark's native scheduler: ~1500 tasks/s), plus single point of failure
- For a 100 ms task, even tens of ms of scheduling+queueing delay is intolerable

Sparrow explores the opposite extreme: many schedulers, *zero* shared state.

Sparrow: Design

Fully decentralized, **stateless** schedulers place tasks by *randomized sampling*:

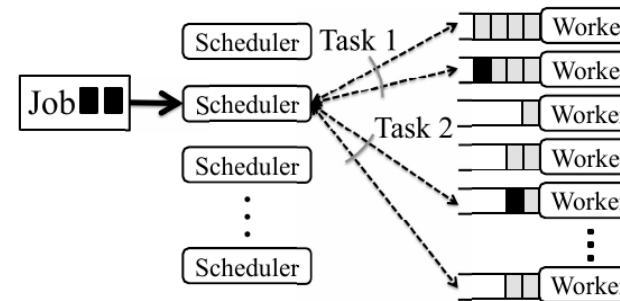
- Start from the classic **power of two choices**: probe 2 random workers, pick the shorter queue → exponentially better than random
- Adapt it to *parallel jobs* with two techniques:
 - **Batch sampling** — sample for the whole job at once
 - **Late binding** — let workers pull tasks when they're actually free

Result: response times within **12% of an ideal omniscient scheduler**, with no scheduler state, trivial scale-out, and ~100 ms failover.

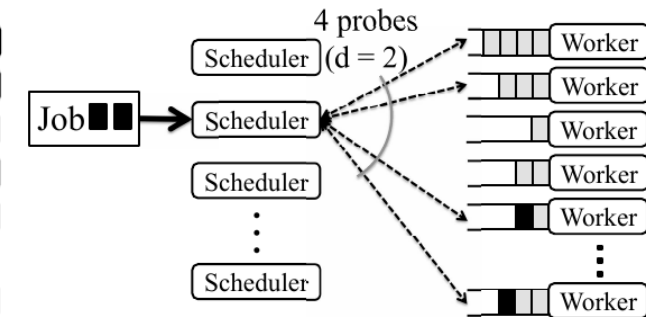
Per-Task vs. Batch Sampling

- Per-task: probe d workers *per task* — but a job waits for its **last** task; tails stay bad
- Batch: probe $d \cdot m$ workers for an m -task job, place tasks on the m **least** loaded of the whole batch (*a lucky probe helps any task*)

(Source: Ousterhout et al., "Sparrow," SOSP 2013, Fig. 2)



(a) Per-task sampling selects queues of length 1 and 3.



(b) Batch sampling selects queues of length 1 and 2.

Late Binding

Two problems remain with sampling on queue lengths:

- **Queue length $\neq$ wait time:** one 300 ms task "looks shorter" than two 50 ms tasks
- **Races:** two schedulers probe the same idle worker and both place tasks there

Late binding: probes place a *reservation*. When the reservation reaches the front of a worker's queue, the worker asks the scheduler for a task; the job's m tasks go to the **first m workers to ask**; the rest get no-ops.

- Tasks bind to the workers that can run them *soonest* — based on actual availability, not stale estimates
- Cost: worker idles for one RTT while requesting — ~2% for 100 ms tasks at 1 ms RTT

Sparrow: Policies Without Central State

- **Per-job constraints** (e.g. "needs GPU"): sample only among satisfying workers
- **Per-task constraints** (data locality): fall back to per-task sampling + late binding [near-optimal since even an ideal scheduler has only ~ 3 choices (replicas) per task]
- **Strict priorities**: per-priority queues at each worker (no preemption)
- **Weighted fair sharing**: per-user weighted fair queueing at each worker $\rightarrow$ cluster-wide fairness *in expectation*
- **Fault tolerance**: failover to a new scheduler in less than 120ms

Part 6: Borg

Large-Scale Cluster Management at Google

Verma et al., Google — EuroSys 2015

Borg: Motivation

Borg is the system that runs essentially **all of Google**: hundreds of thousands of jobs, tens of thousands of machines per cell

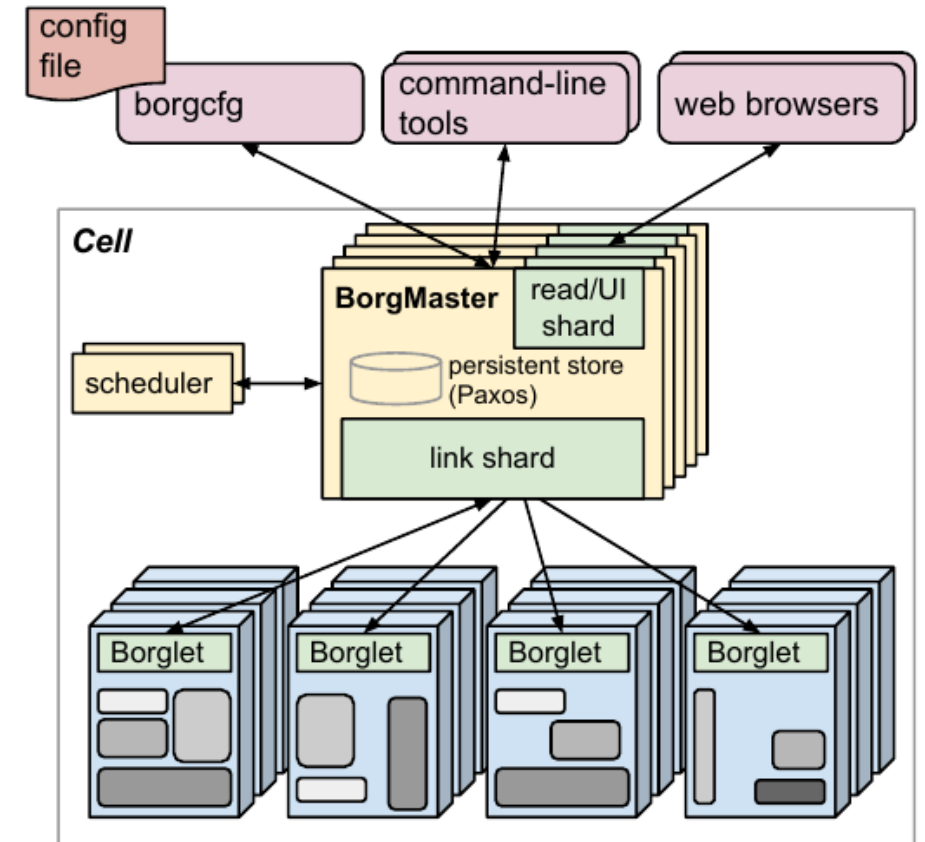
- **Prod** (production): long-running latency-sensitive services (Gmail, Search, BigTable) — must "never" go down
- **Non-prod**: batch jobs (seconds to days)
- Goals: hide resource management & failure handling from developers; very high reliability; scale
- "A few percentage points of utilization = millions of dollars"

Borg: Concepts

- **Cell**: a set of machines that are managed as a unit
- **Job**: name + owner + constraints + n **tasks**; fine-grained requirements: milli-cores of CPU, bytes of RAM; **no fixed-size slots**
- **Alloc**: reserved resources on a machine that outlive individual tasks (e.g. web server + log-saver side task) → the ancestor of the Kubernetes *pod*
- **Priority bands**: monitoring > production > batch > best-effort; higher priority can preempt lower (but prod never preempts prod — no cascades)
- **Quota** checked at *admission* (priced: higher priority costs more; prod quota capped at real cell capacity)

Borg Architecture

- **Borgmaster:** logically centralized, replicated 5× with Paxos; elected leader mutates state; failover ~10 s
- **Borglet** per machine; the master *polls* Borglets (rate control) via stateless **link shards** that diff-compress state
- Tasks keep running even if **all** Borgmaster replicas fail
- **Fauxmaster:** replays checkpoints through real Borgmaster code — simulation for debugging & capacity planning



(Source: Verma et al., "Borg," EuroSys 2015, Fig. 1)

The Borg Scheduler

Async scan of the pending queue, priority order (round-robin within priority):

- **Feasibility check:** which machines satisfy constraints & have enough resources (incl. resources freeable by evicting lower priority)?
- **Scoring:** minimize preemptions; prefer machines that already have the task's packages (startup median 25 s, 80% of it package install); spread across failure domains; mix priorities for load spikes
 - Hybrid of "worst fit" (spread) and "best fit" (pack): 3–5% better packing than best fit

If needed: **preempt** lower-priority tasks (they go back to the pending queue).

Scalability

Techniques that substitute for architectural decentralization:

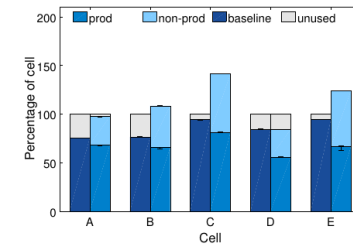
- **Score caching:** reuse feasibility/scores until something relevant changes
- **Equivalence classes:** score once per group of identical tasks
- **Relaxed randomization:** examine machines in random order until enough feasible ones are found ($\approx$ Sparrow's sampling, inside one scheduler!)
- Omega-style **optimistic concurrency** adopted: scheduler runs against a cached cell state; master rejects stale placements $\rightarrow$ multiple schedulers
- 10,000+ task arrivals/min; Borgmaster: 10–14 cores, ≤ 50 GiB; **99.99% availability**

Cell Sharing

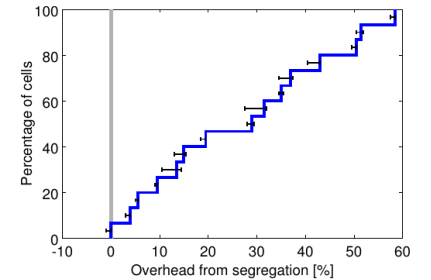
Cell compaction: the smallest cell the real workload packs into (Fauxmaster on production checkpoints of 15 cells).

- Segregating prod & non-prod would need **20–30% more machines** (median cell): prod reserves for rare spikes; Borg runs non-prod in the gap
- Per-team private cells: 2–16× more cells, 20–150% more machines
- CPU interference from sharing: only ~3% worse CPI (low interference)

(Source: Borg, EuroSys 2015, Fig. 5)



(a) The left column for each cell shows the original size and the combined workload; the right one shows the segregated case.

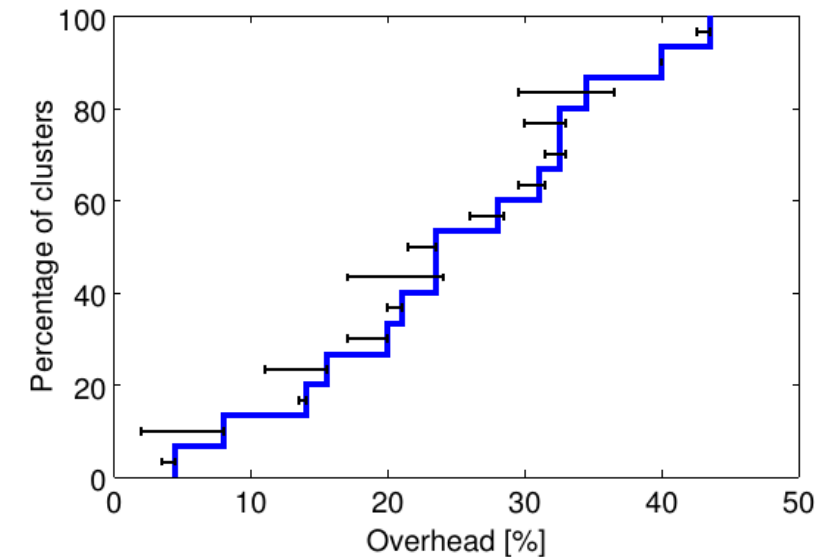


(b) CDF of additional machines that would be needed if we segregated the workload of 15 representative cells.

Borg: Resource Reclamation

- Users over-request; a task's **limit** is rarely used in full
- Borg estimates each task's **reservation** (usage + safety margin)
- Prod is scheduled against **limits** (never reclaimed space); non-prod is packed into the **reclaimed** gap
- If a machine runs out: throttle or kill **non-prod**, **never prod**
- ~**20%** of the workload runs in reclaimed resources (median cell)

(Source: Borg, EuroSys 2015, Fig. 10)



Summary

	Quincy	Delay Sched.	Mesos	Omega	Sparrow	Borg
Architecture	monolithic	monolithic	two-level	shared-state	distributed	monolithic+
State	global	global	offers only	full replicas	none	global
Concurrency	serialized	serialized	pessimistic	optimistic	none needed	optimistic (later)
Locality via	flow costs	waiting	framework	scheduler-specific	per-task sampling	package affinity
Fairness via	capacities	fair share + delay	DRF/offers	caps + audit	per-worker WFQ	priority + quota
Preemption	yes (core)	kill timeouts	revocation	precedence	no	yes (bands)
Gang sched.	no	no	hoarding	all-or-nothing txn	no	no
Target	Dryad batch	Hadoop batch	mixed frameworks	batch + service	sub-second tasks	everything

Recurring Themes

- **Fairness vs. locality:** Quincy optimizes it globally; delay scheduling sidesteps it by exploiting short tasks — *know your workload before you optimize*
- **Kill vs. wait:** preemption gives instant fairness but wastes work; waiting is nearly free when slot turnover is fast
- **Who holds the state?** one scheduler (Quincy/Borg) → resource offers (Mesos) → replicated shared state (Omega) → no state at all (Sparrow)
- **Pessimism vs. optimism:** locks are safe but serialize; optimism wins while conflicts are rare

References

- M. Isard et al., "Quincy: Fair Scheduling for Distributed Computing Clusters," *SOSP 2009*
- M. Zaharia et al., "Delay Scheduling: A Simple Technique for Achieving Locality and Fairness in Cluster Scheduling," *EuroSys 2010*
- B. Hindman et al., "Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center," *NSDI 2011*
- M. Schwarzkopf et al., "Omega: Flexible, Scalable Schedulers for Large Compute Clusters," *EuroSys 2013*
- K. Ousterhout et al., "Sparrow: Distributed, Low Latency Scheduling," *SOSP 2013*
- A. Verma et al., "Large-Scale Cluster Management at Google with Borg," *EuroSys 2015*

(All figures in this deck are reproduced from the respective papers.)